

Structure ÷

Structure is a way to group variables.
Structure can be a collection of dissimilar elements.
Defining structure is creating a custom data type.

Data Type -

Primitive

int

char

float

double

user-defined

data-type

or

Secondary

data-type

or

Custom data-type

or

Derived data-type

Non-Primitive

ex -

Book

date

Student

Employee

Customer

Account

Defining structure

Creating user-defined data-type

Keyword



Struct Tagname → Name of the structure given by you.

// Declare variables

};

Examples ÷

struct student

{

char name[20];

int rollno;

float marks;

};

struct Book

{

int bookid;

char title[20];

double price;

};

struct date

{

int d, m, y;

};

We can't make an array of date even they have data type (int) but they don't have same nature. (d m y) → years

↓ month

Day

Structure Variable declaration ÷

struct can be global or local

struct date

{

int d, m, y;

} d1, d2;

struct date

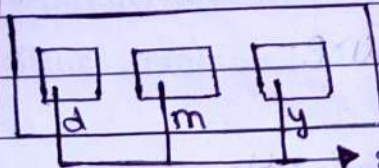
{

int d, m, y;

};

struct date d1, d2;

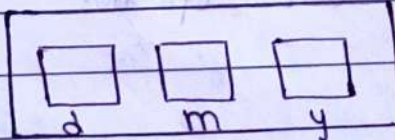
d1 → structure variable



12 bytes

→ structure member variable

d2



12 bytes

Initializing structure member variable during declaration

struct date

{

int d, m, y;

} d1 = { 28, 9, 2024 };

struct date

{

int d, m, y;

};

struct date d1 = { 28, 9, 2024 };

↓
Declare variable in same sequence as the structure member variable are

struct date

{

int d = 28;

int m = 9;

int y = 2024;

} d1;

It is not valid because member variables do not have their separate identity.

- If the number of initializers is less than the number of members then the remaining members are initialized with zero.

Accessing Members of a Structure

Structure Variable • member variable

↳ member access operator

- struct ki maddat se kio bhi data type bnta hai toh osse use krte wakt hmme struct lgana padhta hai

Initializing structure member variables after declaration of structure variable

```
struct date d1;
```

```
d1.d = 29;
```

```
d1.m = 9;
```

```
d1.y = 2022;
```

Taking Input from user :-

```
#include <stdio.h>
```

```
struct date
```

```
{
```

```
    int d;
```

```
    int m;
```

```
    int y;
```

```
};
```

```
int main ()
```

```
{
```

```
    struct date d1;
```

```
    printf ("Enter date : ");
```

```
    scanf ("%d %d %d", &d1.d, &d1.m, &d1.y);
```

```
    printf ("%d-%d-%d", d1.d, d1.m, d1.y);
```

```
    return 0;
```

```
}
```

Output :-

Enter date : 29 9 2024

29-9-2024

* But if we entered
Enter Date : 29/9/2022 → Because scanf by-default considers
29-1-76791904 delimiter to space tab in

To take delimiter of your choice use that delimiter in
scanf between %d.

Eg :
struct date d1;
printf("Enter date:");
scanf("%d/%d/%d", &d1.d, &d1.m, &d1.y);
printf("%d-%d-%d", d1.d, d1.m, d1.y);

Output :
Enter date : 29/9/2024
29-9-2024

Returning a structure variable data from a function ÷

```
#include <stdio.h>
```

```
struct date
```

```
{
```

```
    int d;
```

```
    int m;
```

```
    int y;
```

```
};
```

```
struct date inputDate(); → Declaration of function
```

```
int main()
```

```
{
```

```
    struct date d1;
```

```
    d1 = inputDate(); → function call
```

```
    printf("%d-%d-%d", d1.d, d1.m, d1.y);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

```
struct date inputDate() → Defination of function
```

```
{ → return Data type      → Take nothing Return something
```

```
    struct date somedate;
```

```
    print("In Enter Date :");
```

```
    scanf("%d/%d/%d", &somedate.d, &somedate.m, &somedate.y);
```

```
    return somedate;
```

```
}
```

function call by passing structure

```
#include <stdio.h>
```

```
struct Date
```

```
{
```

```
    int d;
```

```
    int m;
```

```
    int y;
```

```
};
```

```
void showdate(struct Date newDate);
```

```
int main()
```

```
{
```

```
    struct Date d1;
```

```
    printf("Enter a date");
```

```
    scanf("%d/%d/%d", &d1.d, &d1.m, &d1.y);
```

```
    showdate(d1);
```

→ There is no need to define data-type while passing actual argument

```
    return 0;
```

```
}
```

```
void showdate(struct Date newDate)
```

```
{
```

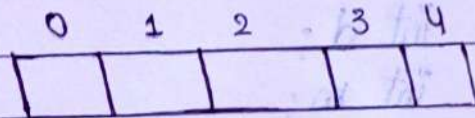
```
    printf("%d-%d-%d", newDate.d, newDate.m, newDate.y);
```

```
}
```

Structure Array ÷

simple array ÷

```
int a[5];
```



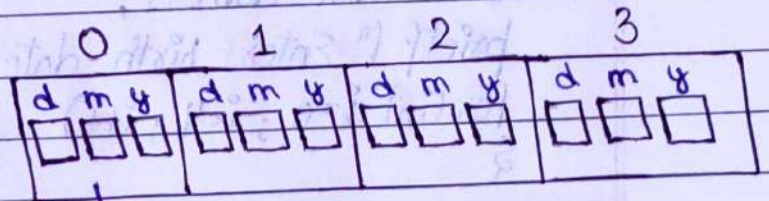
struct array ÷

```
struct date
```

```
{
```

```
    int d, m, y;
```

```
};
```



```
int main()
```

```
{
```

```
    struct date dob[4];
```

Total = 48 bytes

Array of structure :

```
#include <stdio.h>
```

```
struct date
```

```
{
```

```
    int d;
```

```
    int m;
```

```
    int y;
```

```
};
```

```
int main()
```

```
{
```

```
    struct date dob[4];
```

```
    printf("Enter birth date of four person :");
```

```
    for(int i=0 ; i<4 ; i++)
```

```
    {
```

```
        printf("Enter dob :");
```

```
        scanf("%d/%d/%d", &dob[i].d, &dob[i].m, &dob[i].y);
```

```
    }
```

```
    for(int i=0 ; i<4 ; i++)
```

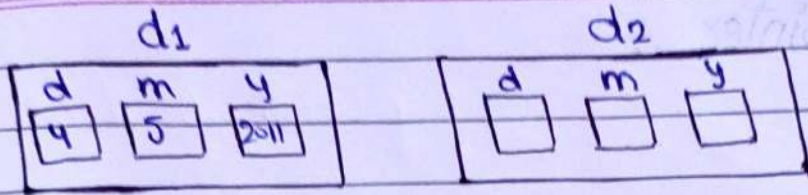
```
    {
```

```
        printf("\n %d-%d-%d", dob[i].d, dob[i].m, dob[i].y);
```

```
    }
```

```
}
```

eg:



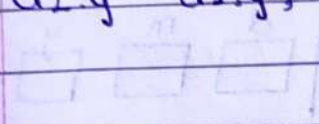
To copy d1 data in d2

$d2.d = d1.d;$

$d2.m = d1.m;$

$d2.y = d1.y;$

or $d2 = d1;$ if d1 and d2 has same data type



$z = h(q)$

$e = m(q)$

$z = h(h)$

$e = m(h)$

$z = h \leftarrow q$

$e = m \leftarrow q$

inverse map

Structure Pointer :

```
#include <stdio.h>
```

```
struct date
```

```
{
```

```
    int d;
```

```
    int m;
```

```
    int y;
```

```
};
```

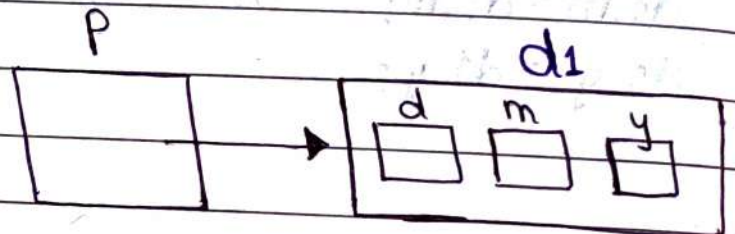
```
int main()
```

```
{
```

```
    struct date d1;
```

```
    struct date *p;
```

```
    p = &d1;
```



Accessing Member variable through Pointer

```
d1.d = 5;
```

```
(*p).d = 5;
```

```
d1.m = 2;
```

```
(*p).m = 2;
```

- $P \rightarrow d = 5;$
 $P \rightarrow m = 2;$

$\rightarrow$ composing sign
 $\downarrow$
 minus sign

Type defined structure

Type def → it is used to create alias of old data type.

```
#include <stdio.h>
```

```
struct student
```

```
{
```

```
    int rollno;
```

```
    char name[40];
```

```
    float marks;
```

```
};
```

```
int main()
```

```
{
```

```
    struct student s = { 10, "Gaurav", 80.9 };
```

```
    printf ("%d %s %f", s.rollno, s.name, s.marks);
```

```
}
```

```
#include <stdio.h>
```

```
typedef struct student
```

```
{
```

```
    int rollno;
```

```
    char name[40];
```

```
    float marks;
```

```
} stu;
```

```
int main()
```

```
{
```

```
    stu s = { 10, "Gaurav", 80.9 };
```

```
    printf ("%d %s %f", s.rollno, s.name, s.marks);
```

```
}
```

Structure Padding in C ÷

When an object of some structure type is declared then some continuous block of memory will be allocated to structure members.

Ex ÷ struct abc

{

char a ;

char b ;

int c ;

} var ;

Calculating the size of the struct ÷

struct abc

{

char a ; // 1 byte

char b ; // 1 byte

int c ; // 4 bytes

} 6 bytes X

} var ;

Processor doesn't read 1 byte at a time from memory.

It reads 1 word at a time.

If we have a 32-bit processor then it means it can access 4 bytes at a time.

And if we have a 64-bit processor then it means it can access 8 bytes at a time.

struct abc

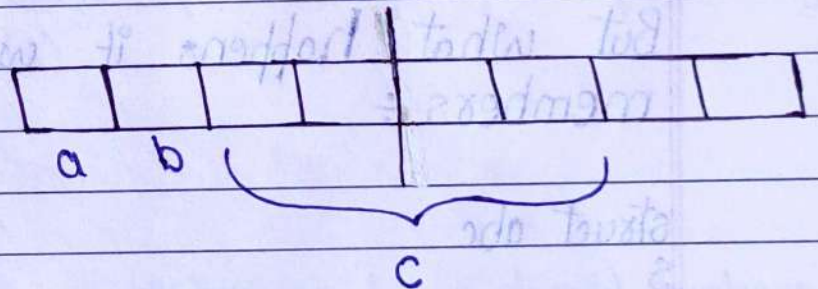
{

char a ; // 1 byte

char b ; // 1 byte

char c ; // 4 bytes

};



In 1 CPU cycle, char a, char b and 2 bytes of int c can be accessed. But whenever we want the value stored in variable c, 2 cycles are required to access the contents of variable c. In first cycle, 1st 2 bytes can be accessed and in 2nd cycle, last 2 bytes.

It's an unnecessary wastage of CPU cycles.

*

Here we can save the number of cycles by using the concept called padding.

Due to the concept of Padding ÷

struct abc

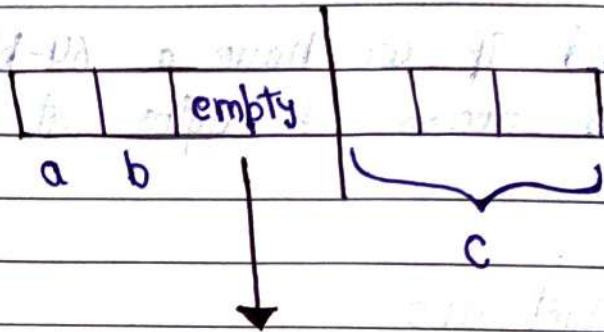
{

char a; // 1 byte

char b; // 1 byte

int c; // 4 byte

};



Due to the concept of Padding empty spaces are created here

Padding is done automatically by the compiler.

But what happens if we change the order of members ÷

struct abc

{

char a; // 1 byte

int b; // 4 bytes

char c; // 1 byte

};

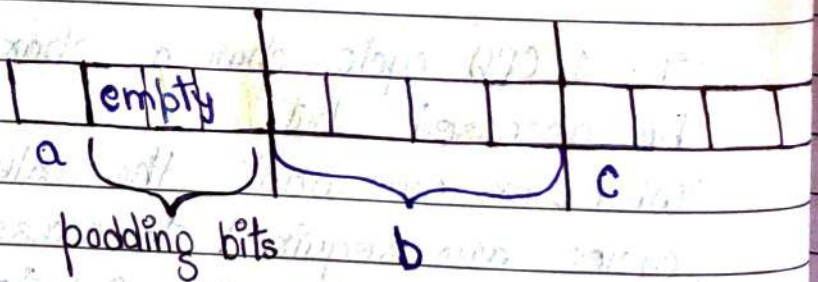
int main()

{

struct abc var;

printf("%d", sizeof(var)); // 12

}



Structure Packing

Because of structure padding, size of the structure becomes more than the size of the actual structure. Due to this some memory will get wasted.

But CPU cycle doesn't get wasted means the speed of CPU increases on the penalty of memory.

So to void the wastage of memory we can use the concept of structure packing.

- We can avoid the wastage of memory by simply writing `#pragma pack(1)`

```
#pragma pack(1)
```

```
struct abc
```

```
{
```

```
    char a;
```

```
    int b;
```

```
    char c;
```

```
}
```

```
int main
```

```
{
```

```
    struct abc var;
```

```
    printf("%d", sizeof(var)); // 6
```

```
}
```

`#pragma` is a special purpose directive used to turn on or off certain features.

- Here CPU cycle is wasted